Project - Low Level Design

On

Marketing Planner Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	YASH YADAV	EN22CS3011113
2	AYUSH MISHRA	EN22EL301020
3	VISHAL	EN22CS3011091
4	ASHUTOSH KUMAR	EN22EL301019
5	HARSH MANDLOI	EN22ME304035
6	AYUSH MODI	EN21CS301196

Group Name: 13D10

Project Number: GAI-26

Industry Mentor Name: Mr. Ashwin

University Mentor Name: Prof Maya Yadav Baniya

Academic Year: 2025-2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.
2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.
3. References

1. Introduction

The Marketing Planner Agent is an AI-powered full-stack application that enables marketing professionals and teams to generate structured, validated execution plans from high-level marketing goals. The system leverages the LLaMA 3.3-70b versatile large language model via the Groq inference API to decompose goals into actionable subtasks, validates them against mock resource constraints, and returns an ordered execution schedule with dependency tracking.

The application is built on a React and TypeScript frontend deployed on Vercel, and a Python FastAPI backend deployed on Render. Authentication is handled via Firebase Authentication supporting Google sign-in, GitHub sign-in, and email and password registration.

1.1. Scope of the Document

This Low Level Design document describes the internal architecture, component interactions, data flows, and API specifications of the Marketing Planner Agent system. It covers:

- The overall application architecture and deployment topology
- Detailed component responsibilities and interactions
- The request and response lifecycle from user input to plan generation
- The authentication and route protection mechanism
- All API endpoints with request and response schemas
- Key design decisions including error handling, environment configuration, and CORS policy

This document does not cover infrastructure provisioning, CI/CD pipeline configuration, or future planned features such as vector database integration.

1.2. Intended Audience

This document is intended for the following readers:

Audience	Purpose
University Mentors and Evaluators	Assessment of system design quality and technical depth
Industry Mentors	Review of architectural decisions and implementation approach
Developers	Understanding component responsibilities for future development
Technical Reviewers	Validation of design against project requirements

1.3. System Overview

The Marketing Planner Agent is a multi-tier web application with the following high-level components:

Layer	Technology	Responsibility	Hosting
Presentation	React 18, TypeScript, Tailwind CSS, Vite	User interface, routing, authentication UI	Vercel
API Gateway	FastAPI, Uvicorn, Python 3.11	HTTP endpoints, request validation, CORS	Render
Agent Layer	LangChain, Groq API, LLaMA 3.3-70b	Goal decomposition, task parsing	Render
Validation Layer	Python functions	Budget, team, timeline validation	Render
Scheduling Layer	Python functions	Dependency resolution, schedule generation	Render
Authentication	Firebase Authentication	Google, GitHub, Email/Password sign-in	Firebase Cloud

The system follows a client-server architecture where the React frontend communicates exclusively with the FastAPI backend via a single REST endpoint. All AI processing, tool validation, and scheduling logic resides on the backend. The frontend is responsible only for rendering results and managing user authentication state via Firebase.

2. System Design

2.1. Application Design

The application uses a three-tier system with clear separation between presentation, logic, and external services.

- Frontend: A single-page application (SPA) with three primary routes: / (Landing), /login (Auth), and /planner (Protected).
- Backend: A FastAPI application exposing two HTTP endpoints that orchestrate the agent and tools in a sequential pipeline.

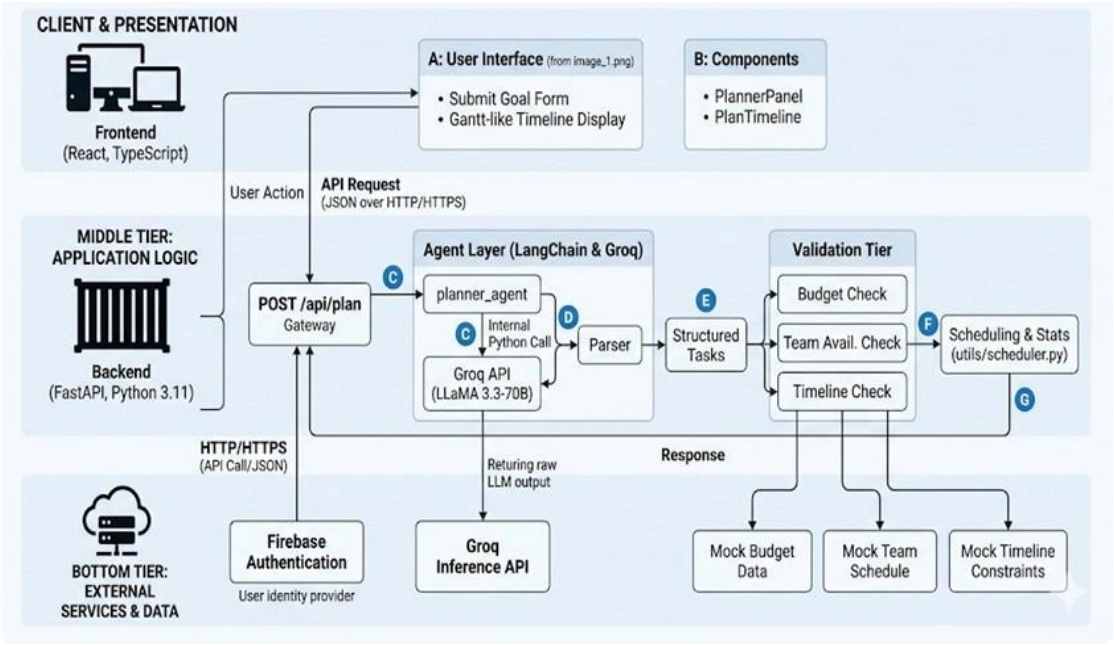


Fig.: 1 Application Design

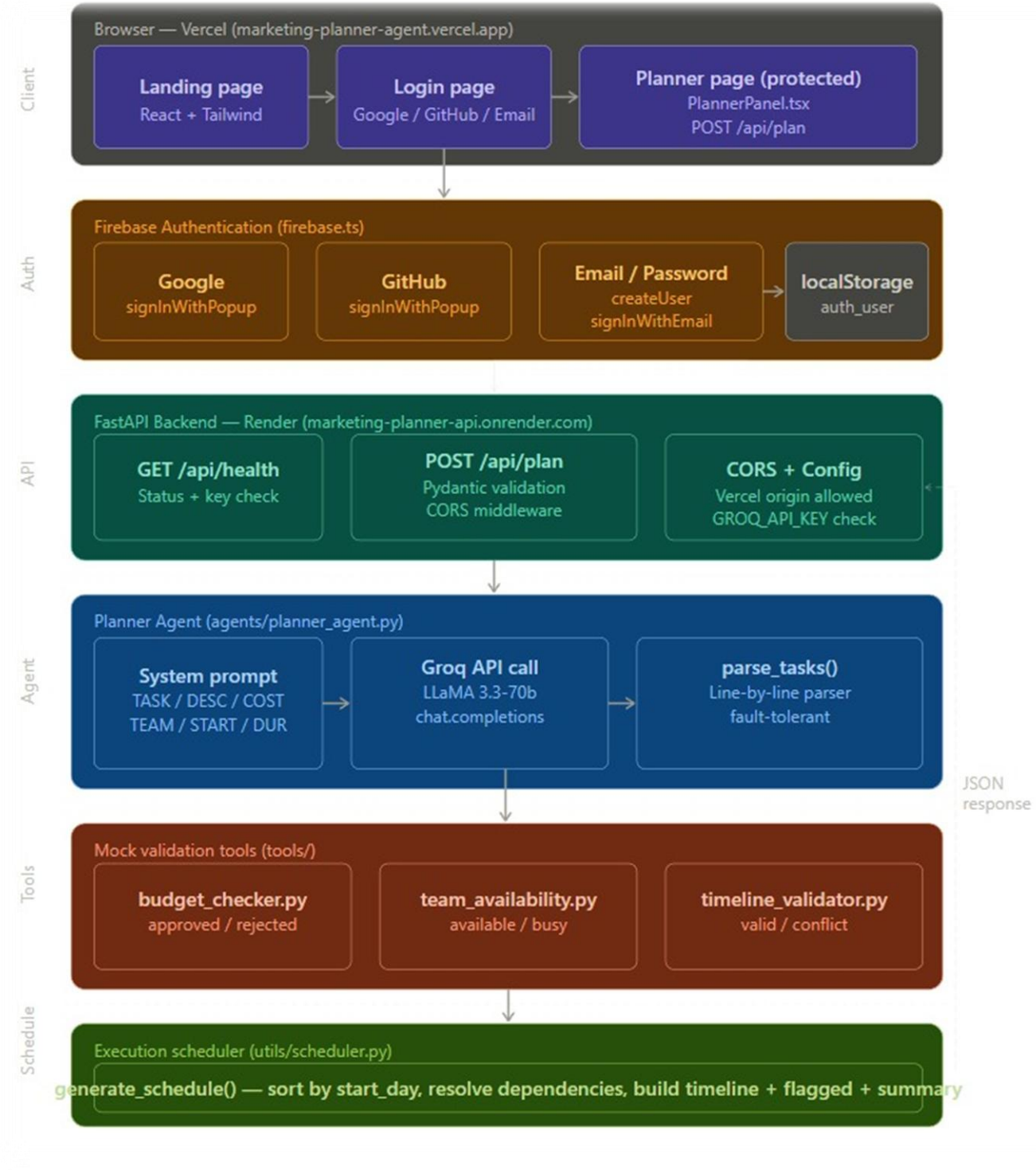
2.2. Process Flow

2.2.1. Authentication Flow

Users authenticate via Firebase using Google, GitHub, or Email. Upon success, the user's email is stored in localStorage as auth_user, which the /planner route checks to allow access.

2.2.2. Plan Generation Flow

1. Input: User submits a marketing goal via the PlannerPanel.
2. Request: A POST request is sent to the FastAPI backend.
3. Agent: The agent calls Groq (LLaMA 3.3) to generate structured subtasks.
4. Parsing: The raw text is parsed line-by-line using specific prefixes (TASK, COST, etc.).
5. Validation: Each task is checked sequentially against Budget, Team, and Timeline tools.
6. Scheduling: Validated tasks are sorted by start day and dependencies are resolved.
7. Output: The final JSON schedule is rendered as a timeline on the frontend.


Client
Browser — Vercel (marketing-planner-agent.vercel.app)
Landing page
React + Tailwind
Login page
Google / GitHub / Email
Planner page (protected)
PlannerPanel.tsx
POST /api/plan
Auth
Firebase Authentication (firebase.ts)
Google
signInWithPopup
GitHub
signInWithPopup
Email / Password
createUser
signInWithEmail
localStorage
auth_user
API
FastAPI Backend — Render (marketing-planner-api.onrender.com)
GET /api/health
Status + key check
POST /api/plan
Pydantic validation
CORS middleware
CORS + Config
Vercel origin allowed
GROQ_API_KEY check
Agent
Planner Agent (agents/planner_agent.py)
System prompt
TASK / DESC / COST
TEAM / START / DUR
Groq API call
LLaMA 3.3-70b
chat.completions
parse_tasks()
Line-by-line parser
fault-tolerant
JSON
response
Tools
Mock validation tools (tools/)
budget_checker.py
approved / rejected
team_availability.py
available / busy
timeline_validator.py
valid / conflict
Schedule
Execution scheduler (utils/scheduler.py)
generate_schedule() — sort by start_day, resolve dependencies, build timeline + flagged + summary

2.3. Information Flow

From	To	Data	Format
Browser	FastAPI	User goal	JSON POST body
Agent	Groq API	System prompt + goal	Chat completion
Groq API	Parser	Structured text	Plain text
Scheduler	FastAPI	Validated schedule dict	Python dict
FastAPI	Browser	Final PlanSchedule	JSON

2.4. Components Design

- api/app.py: Initializes FastAPI, configures CORS, and manages /api/plan endpoints.
- agents/planner_agent.py: Manages the system prompt and LLM integration.
- Validation Tools: Stateless functions (check_budget, check_team_availability, validate_timeline) that use mock data to verify task feasibility.
- utils/scheduler.py: Assembles the timeline and computes summary statistics like total budget and completion rate.

2.5. Key Design Considerations

- LLM Parsing: Uses line-by-line string matching rather than JSON to avoid errors from malformed LLM outputs.
- Environment Variables: Backend uses .env for Groq keys; frontend uses web/.env with VITE_ prefixes for Firebase and API URLs.

- Mock Data: Current validation tools use hardcoded dictionaries, which can be replaced by real APIs in production.
- Render Cold Starts: Due to the free tier, the first request may take up to 50 seconds after inactivity.

2.6. API Catalogue

POST /api/plan

- Description: Accepts a goal and returns a validated schedule.
- Request Body: { "goal": "string" }.
- Key Response Fields: generated_at, timeline (array of tasks), flagged (failed tasks), and summary (budget, duration).

3. References

- Frameworks: FastAPI, React, LangChain.
- APIs/Models: Groq API (LLaMA 3.3-70B), Firebase Authentication.
- Deployment: Vercel (Frontend), Render (Backend).